The accuracy calculation function (fig. 7) indicates that the model performs regression on the output to match the ground truth digits. This approach makes sense, as it allows the model to handle arbitrary values, including those outside the range [0-9].

```
# Model
class CompositionalModel(nn.Module):
    def __init__(self, vocab_size, hidden_size=64):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_size)
        self.lstm = nn.LSTM(hidden_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, 1)

    def forward(self, x):
        embedded = self.embedding(x)
        lstm_out, _ = self.lstm(embedded)
        hidden = lstm_out[:, -1, :]
        return self.fc(hidden)

    def get_compositional_loss(self, hidden_states):
        return torch.mean((hidden_states[:, 1:] - hidden_states[:, :-1]).pow(2))
```

Figure 5 | The generated model class shows an embedding layer, a single LSTM layer, and a linear layer head.

```
for epoch in range(n_epochs):
    model.train()
    train_loss = 0
    comp_loss = 0

    for batch_idx, (expr, result) in enumerate(train_loader):
        expr, result = expr.to(device), result.to(device)

        optimizer.zero_grad()
        output = model(expr)

        # Calculate main loss
        loss = criterion(output.squeeze(), result)

        # Add compositional regularization
        hidden_states = model.embedding(expr)
        comp_reg = model.get_compositional_loss(hidden_states)
        total_loss = loss + compositional_weight * comp_reg

        total_loss.backward()
        optimizer.step()

        train_loss += loss.item()
        comp_loss += comp_reg.item()
```

Figure 6 | The generated training loop shows the loss function as well as the proposed regularization.

```
with torch.no_grad():
    for expr, result in test_loader:
        expr, result = expr.to(device), result.to(device)
        output = model(expr)
        test_loss += criterion(output.squeeze(), result).item()

        # Calculate accuracy within a tolerance
        correct += torch.sum(torch.abs(output.squeeze() - result) < 0.5).item()
        total += result.size(0)
```

Figure 7 | The generated accuracy calculation function uses regression to match an output with a ground truth.